Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
Ingeniería en Ciencias y Sistemas

Sistemas Operativos 2

Manual técnico de REDXela

Arquitectura y decisiones de implementación

27 de octubre de 2026

Propósito y alcance

REDXela es una aplicación de escritorio que simula la recepción, almacenamiento y preparación de pedidos con recursos limitados. Este manual describe la implementación Java entregada, sus mecanismos de concurrencia y las decisiones necesarias para mantener coherencia entre pedidos, stock y recursos.

El sistema utiliza un proceso de la JVM con varios hilos. La comunicación entre áreas se realiza mediante una cola dentro de ese proceso. No existen servidor HTTP, base de datos ni procesos del sistema operativo separados para cada área.

Contenido

Entorno y arquitectura · páginas 2 y 3

Modelo de datos y catálogo · páginas 4 y 5

Motor y sincronización · páginas 6 a 8

Recepción e inventario · páginas 9 y 10

Almacenamiento y planificación · páginas 11 y 12

Tiempo y objetivos de servicio · página 13

Deadlock y recuperación · páginas 14 y 15

Generador y comparación · páginas 16 y 17

Métricas e interfaz · páginas 18 y 19

Pruebas y límites · páginas 20 y 21

Entorno de ejecución y construcción

El artefacto Maven se identifica como gt.redxela:redxela:1.0-SNAPSHOT. El proyecto compila para Java 17 y conserva el estado en memoria. Se necesita una sesión gráfica para abrir JavaFX; un entorno sin escritorio puede revisar el código y ejecutar pruebas de servicios si dispone del JDK y Maven.

Componente	Versión o configuración
Java	release 17; codificación UTF-8
JavaFX Controls y FXML	17.0.14
JUnit Jupiter	5.10.2; dependencia de pruebas
Maven Compiler Plugin	3.13.0
Maven Surefire Plugin	3.2.5; useModulePath=false
Maven JAR Plugin	3.4.1
JavaFX Maven Plugin	0.0.8

```
mvn clean verify
mvn javafx:run
mvn package
```

verify compila y ejecuta las pruebas; javafx:run abre gt.redxela.ui.RedXelaApp; package construye el JAR. El JAR declara la clase principal, pero no empaqueta las dependencias ni bibliotecas nativas JavaFX. El procedimiento soportado para iniciar la interfaz es Maven, no asumir que java -jar basta.

Configuración temporal

```
REDXELA_ESCALA_TIEMPO=60 mvn javafx:run
```

El factor se lee al construir cada reloj. La propiedad Java redxela.escalaTiempo tiene precedencia sobre la variable de entorno. Se aceptan factores finitos de 0.01 a 1000. Un factor 60 convierte un segundo real en un minuto simulado para envejecimiento y objetivos de espera.

Referencia de implementación: pom.xml; RedXelaApp.start; EscalaTiempoSimulado.configurada.

Arquitectura y dependencias

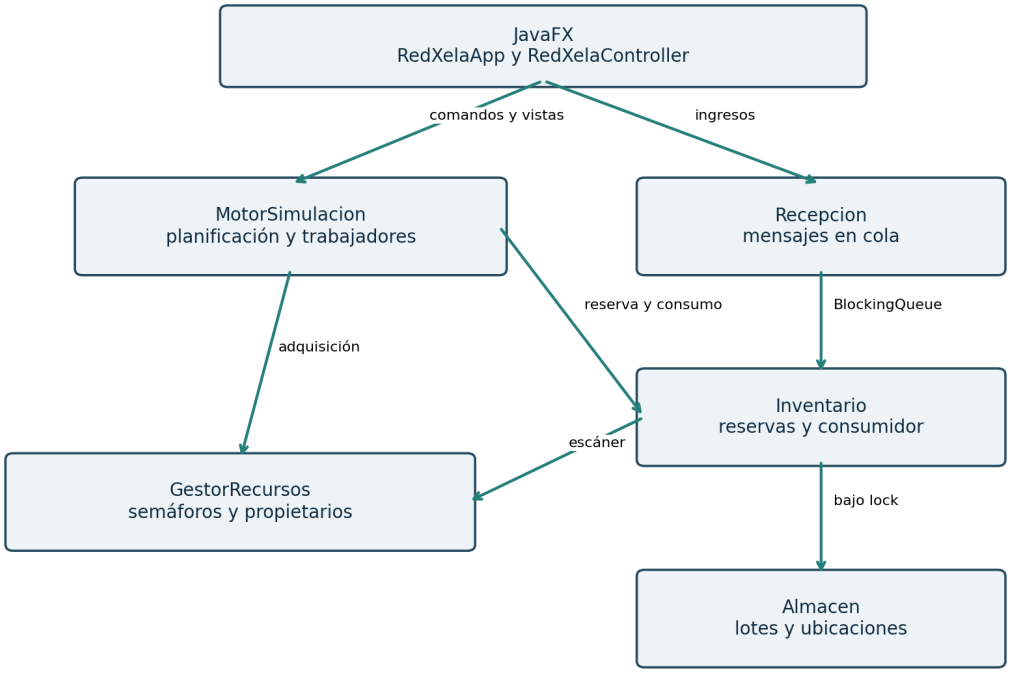


Diagrama 1. Componentes del modo interactivo y sus dependencias.

RedXelaController compone los servicios y conecta Inventario con MotorSimulacion. El motor decide qué pedido despachar; Inventario decide si existe stock reservable; GestorRecursos administra estaciones y equipo. Almacen mantiene la distribución física lógica y es utilizado bajo el lock de Inventario.

Los paquetes modelo, planificacion, concurrencia, inventario, almacenamiento y simulacion no importan JavaFX. La interfaz consume snapshots de esos servicios. Esto permite probar el dominio sin abrir una ventana y crear motores independientes para las comparaciones.

El FXML define la estructura principal y el controlador construye programáticamente las páginas. CSS establece la presentación. El diseño separa servicios de la interfaz, aunque la composición y buena parte de la vista están concentradas en un controlador grande.

Referencia de implementación: RedXelaController campos de composición; MotorSimulacion.conectarInventario; redxela.fxml.

Modelo de dominio y estado operativo

Tipo	Responsabilidad
Cliente	Record inmutable con UUID, nombre, correo normalizado y servicio por defecto.
Pedido	Record con UUID, cliente, nivel contratado, líneas, estado inicial y fecha Instant.
LineaPedido	Producto canónico, tipo de mercancía y cantidad positiva.
Producto	ID, nombre, tipo y volumen entero por unidad.
RegistroClientes	Índices por UUID y correo protegidos por ReentrantReadWriteLock.
RuntimeOrder	Estado mutable privado del motor, marcas de tiempo, worker, consumo y error.

Pedido valida que existan líneas, que no sean nulas y que la suma de cantidades no desborde un int. totalUnidades usa Math.addExact. LineaPedido exige que productoId coincida con el nombre del tipo; por eso el catálogo actual mantiene un producto por tipo.

El estado efectivo de un pedido no se modifica dentro del record Pedido. MotorSimulacion mantiene RuntimeOrder y publica EstadoPedidoMotor, que combina el pedido original con su estado y métricas vigentes. Consultar solo pedido.estado() no describe su ciclo de vida actual.

Las cuentas se actualizan sustituyendo el record Cliente en el registro. Los pedidos anteriores conservan su referencia y el nivel contratado. El registro comprueba correos duplicados bajo writeLock después de normalizarlos con trim y minúsculas; no implementa contraseña ni autenticación.

Instant sirve como fecha civil de creación. Los intervalos de espera y procesamiento se miden con nanoTime para evitar cambios del reloj del sistema.

Referencia de implementación: Cliente, Pedido, LineaPedido, Producto, RegistroClientes; MotorSimulacion.RuntimeOrder; EstadoPedidoMotor.

Catálogo y requisitos de recursos

El catálogo fijo permite seleccionar productos sin una pantalla de mantenimiento. Los volúmenes son enteros y representan capacidad lógica del almacén, no una unidad física como metros cúbicos.

Producto	Volumen por unidad	E	C	P	M	Q	S
Documentos	1	1	0	1	0	1	1
Frágil	3	1	0	2	0	1	1
Refrigerado	4	1	1	2	0	1	1
Voluminoso	8	1	1	2	1	1	1
Peligroso	5	1	1	1	0	2	1

E: empaque; C: área de carga; P: encargado; M: montacargas; Q: control de calidad; S: escáner. Las capacidades globales son respectivamente 6, 2, 10, 3, 4 y 1.

Las combinaciones representan el manejo previsto: frágiles requieren más personal; refrigerados y voluminosos requieren carga; voluminosos incluyen montacargas; peligrosos requieren dos controles de calidad. El sistema no modela refrigeración física ni procesos industriales especializados.

Combinar varias líneas

NecesidadesPedido.calcular toma el máximo requerido por recurso entre las líneas, no multiplica por cantidad ni suma equipos por producto. Por ejemplo, frágil más voluminoso solicita 1 empaque, 1 carga, 2 encargados, 1 montacargas, 1 calidad y 1 escáner.

Esta decisión considera que el pedido reutiliza una combinación de recursos durante su preparación. La cantidad de artículos aumenta la duración del trabajo. Los recursos permanecen retenidos durante todo ese trabajo; no se dividen en fases independientes de empaque, revisión y despacho.

Referencia de implementación: CatalogoProductos; CatalogoMercancias; TipoMercancia; TipoRecurso; NecesidadesPedido.calcular.

Ciclo de vida del motor

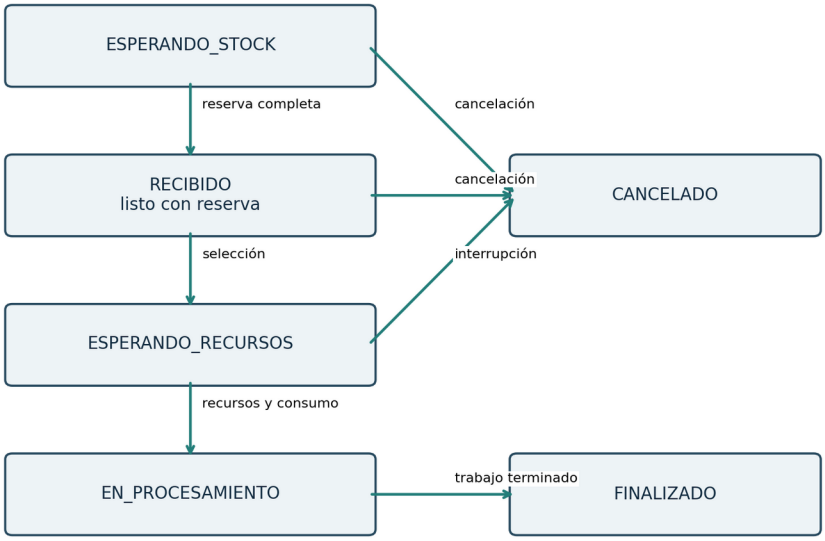


Diagrama 2. Flujo principal. RECIBIDO puede volver a espera de stock al reasignar reservas.

iniciar arranca Inventario y envía tantos bucles workerLoop al ExecutorService como trabajadores configurados. enviar valida requisitos, duplicación de UUID y capacidad estructural del pedido antes de añadirlo a orders y pending.

reevaluarStockLocked ordena los pedidos pendientes según la estrategia y solicita a Inventario una reasignación atómica. Los que reciben reserva pasan a RECIBIDO. takeNext solo selecciona ese estado, elimina el pedido de pending y lo asocia al hilo para impedir doble despacho.

workerLoop adquiere los recursos, consume la reserva, fija inicio y objetivo, ejecuta doWork y marca FINALIZADO. CANCELADO también es accesible desde procesamiento por interrupción o cierre; las transiciones inválidas lanzan IllegalStateException.

Referencia de implementación: MotorSimulacion.iniciar, enviar, reevaluarStockLocked, takeNext, workerLoop, transition.

Hilos y secciones críticas

Componente	Mecanismo y finalidad
Motor	ExecutorService fijo; ReentrantLock y Condition para estados, pending y selección.
Recursos	ReentrantLock justo, Condition y Semaphore por tipo; propietarios y permisos se actualizan juntos.
Inventario	Hilo consumidor dedicado y ReentrantLock para reservas, lotes y snapshots.
Recepción	Métodos synchronized, semáforo de admisión y cola acotada.
Clientes	ReentrantReadWriteLock para índices por ID y correo.
Generador	ScheduledExecutorService de un hilo; LongAdder para conteos.
Interfaz	Hilo JavaFX, Timeline, executor de fondo y hilo dedicado de cierre.
Deadlock	Coordinador, pool de dos operaciones, monitor y CountDownLatch.

El modo interactivo tiene tres workers y una duración de 250 ms reales por unidad. La cola de recepción admite 12 mensajes según la configuración del controlador. La interfaz usa un executor de fondo de un hilo; el número de hilos no crece por cada pedido.

Condition.await libera temporalmente su lock y lo recupera al despertar. Cada espera revisa nuevamente su condición dentro de un while. Las pausas de doWork representan duración de trabajo y no sustituyen la exclusión mutua.

El camino compuesto usa lock del motor seguido del lock de inventario. Inventario invoca cambioStock después de liberar su propio lock; así no conserva ese lock mientras intenta entrar al motor desde un callback. Almacen no tiene lock interno: sus mutaciones deben continuar bajo el de Inventario.

Referencia de implementación: MotorSimulacion; Inventario; Recepcion; GestorRecursos; RegistroClientes; GeneradorPedidos.

Adquisición y liberación de recursos

GestorRecursos crea un pool por TipoRecurso. Cada pool tiene capacidad, Semaphore, contador de disponibles, mapa de propietarios y acumulación de permiso por tiempo. El orden global normal sigue el enum: EMPAQUE, AREA_CARGA, ENCARGADO, MONTACARGAS, CALIDAD y ESCANER.

Algoritmo de adquisición

1. Valida cada cantidad y registra un identificador de reserva que no debe estar activo.

2. Recorre el EnumMap en el orden global. Si falta capacidad, registra la espera y llama a changed.await.

3. Cuando existe capacidad, consume permisos con tryAcquire y actualiza disponibles y propietarios bajo el mismo lock.

4. Devuelve una Reserva AutoCloseable. Si se interrumpe o falla durante una adquisición parcial, devuelve lo obtenido.

El hilo no espera bloqueado dentro de Semaphore.acquire: la espera la coordina Condition junto al registro. El semáforo aporta permisos reales y comprueba consistencia con disponibles. Por esa coordinación adicional no debe prometerse un orden FIFO estricto de atención solo porque el semáforo sea justo.

Reserva.close es idempotente. try-with-resources la ejecuta al salir normalmente o por excepción. updateOccupied suma permisos ocupados multiplicados por el tiempo transcurrido antes de modificar el conteo.

Consecuencia del escáner único

Todos los tipos solicitan un escáner y lo conservan durante doWork. En esta versión, solo un pedido puede estar preparando trabajo a la vez, aunque otros workers puedan estar esperando recursos. Recepción compite por ese mismo escáner. Esto limita el paralelismo efectivo y puede acercar los tiempos totales entre políticas.

Referencia de implementación: GestorRecursos.acquire, releasePartial, updateOccupied, Reserva.close; TipoRecurso; CatalogoMercancias.

Recepción y comunicación entre áreas

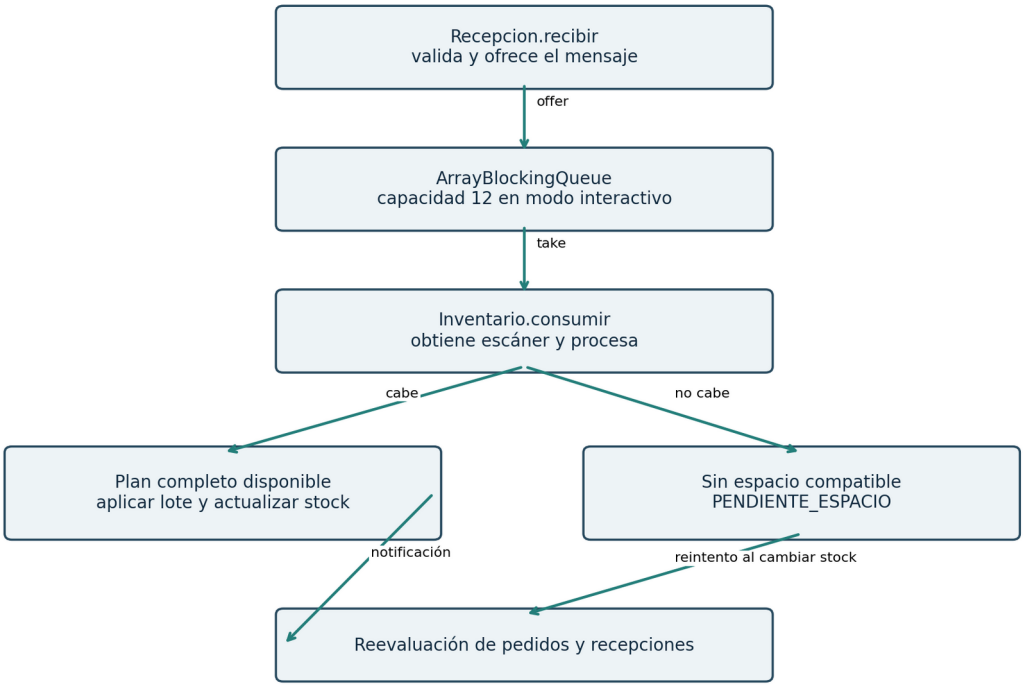


Diagrama 3. Flujo de mensajes y aplicación de recepciones.

Recepcion.recibir valida cantidad y volumen, comprueba cierre y pausa, obtiene un permiso de admisión y usa queue.offer. Una cola saturada devuelve RECHAZADA_COLA_LLENA; no bloquea a la interfaz ni registra una recepción aceptada en silencio.

Inventario.consumir ejecuta queue.take en un hilo dedicado. Obtiene el escáner compartido antes de procesar el mensaje. recepcionesProcesadas evita aplicar dos veces el mismo UUID dentro de la sesión. Esto es comunicación entre hilos del mismo proceso, no IPC mediante sockets o procesos externos.

La toma de mensajes es FIFO. El listado pendiente por espacio se recorre por llegada, pero se omite temporalmente un lote que no cabe para aplicar otros compatibles. Por eso el orden de aplicación física puede diferir del orden de aceptación.

Referencia de implementación: Recepcion.recibir y completar; SolicitudRecepcion; Inventario.consumir, procesar y reevaluarPendientesLocked.

Inventario y reservas atómicas

Inventario deriva las existencias físicas de Almacen.stockPorProducto. No mantiene un contador físico independiente que pueda divergir de los lotes. Las reservas se guardan por UUID de pedido y todas las comprobaciones de una orden se realizan dentro de una misma sección crítica.

```
disponible(producto) = físico(producto) - reservado(producto)
```

1. El motor agrega líneas repetidas por producto con Math.addExact.

2. reasignarReservas libera las reservas de las órdenes que participan en el plan y vuelve a otorgarlas en el orden recibido. Las órdenes ya despachadas no participan.

3. Solo se concede una reserva si hay disponibilidad suficiente para todos los productos. No se retiene stock parcial de una orden sin completar.

4. Al obtener todos los recursos, consumirReserva extrae los lotes, elimina la reserva y registra el pedido como consumido.

5. La liberación física permite reintentar recepciones pendientes y notificar al motor para reevaluar stock.

Cancelar antes del consumo libera la reserva. Cancelar después no repone mercancía: el sistema considera que ya entró a preparación. ordenesConsumidas evita consumo repetido y permite que el detalle de la orden trate sus líneas como satisfechas después de retirar el stock.

disponibilidadPedido calcula disponibilidad global más la reserva propia y limita lo mostrado a lo solicitado. snapshot reúne stock, ubicaciones y lotes bajo el lock de Inventario. Los snapshots separados de motor e inventario pueden representar instantes cercanos, no una transacción global única de toda la pantalla.

Referencia de implementación: Inventario.reservar, reasignarReservas, consumirReserva, liberarReserva, disponibilidadPedido y snapshot.

Asignación física de almacenamiento

Almacen contiene Slot internos y una lista de ParteLote. Cada parte registra loteId, productoId, ubicacionId y cantidad. Ubicacion publica capacidad y volumen utilizado; PlanAlmacenamiento conserva una lista inmutable de partes previstas.

Planificar y aplicar

planificar recorre UB-001 en adelante. Una ubicación es compatible si está vacía o contiene el mismo producto. Calcula las unidades enteras que caben con división de volumen libre entre volumen unitario. Retorna un plan completo o null sin modificar stock. aplicar precomprueba las partes y luego actualiza ubicaciones y lotes bajo el lock externo de Inventario.

extraer recorre las partes en su orden almacenado y descuenta cantidades. Si una ubicación queda con volumen cero, elimina su producto asignado. Un lote puede ocupar ubicaciones no contiguas; la extracción no compacta ni traslada otras existencias.

Validar un pedido en almacén vacío

validarCabeEnAlmacenVacio agrega el volumen total y, si no supera la capacidad, usa canAssign para buscar una asignación de ubicaciones. Ordena capacidades, explora productos y memoriza estados fallidos. Esta búsqueda evita rechazar una combinación factible solo por un orden voraz desfavorable. Con muchas ubicaciones, su costo puede crecer de forma combinatoria.

El espacio compatible en unidades es la suma, por ubicación compatible, de floor(volumen libre / volumen unitario). No equivale a dividir la suma de todo el espacio: una pieza no se divide entre ubicaciones. La regla de un producto por ubicación y los restos no utilizables explican pendientes aun con volumen libre total.

Una reserva completa y consumo al inicio justifican rechazar pedidos que jamás cabrían íntegros. Ese rechazo no implica que un pedido que cabe esté garantizado: otras existencias pueden impedir temporalmente su abastecimiento.

Referencia de implementación: Almacen.planificar, aplicar, extraer, validarCabeEnAlmacenVacio, canAssign y unidadesCompatiblesDisponibles.

Políticas de planificación

Política	Criterio principal	Desempate
Prioridad estricta	Prioridad base descendente	Secuencia de llegada
Prioridad con envejecimiento	Prioridad efectiva descendente	Secuencia de llegada
Menor trabajo primero	Total de unidades ascendente	Secuencia de llegada

Las prioridades internas son Exprés 5, Prioritario 4, Estándar 3, Programado 2 y Económico 1. No son los números de nivel del enunciado: un valor interno mayor gana en las políticas por prioridad.

```
prioridad efectiva = prioridad base
                   + floor(espera simulada / intervalo)
```

Ejemplo: con intervalo 10 segundos simulados, un Económico que esperó 50 segundos tiene prioridad 1 + 5 = 6. Supera a un Exprés recién llegado con prioridad 5. El tiempo se reevalúa al seleccionar; no se usa una PriorityBlockingQueue con prioridades que quedarían obsoletas.

PlanificadorPedidos.ordenar crea una lista ordenada con un instante común. seleccionar toma su primer elemento. El motor emplea estos criterios tanto para planear reservas de stock como para escoger pedidos listos. Cambiar estrategia reevalúa pendientes y no interrumpe trabajo ya despachado.

Las políticas son no expropiativas respecto a pedidos en procesamiento. Menor trabajo primero puede reducir espera media de pedidos pequeños y postergar grandes. Prioridad estricta puede postergar niveles bajos. Envejecimiento mejora oportunidades con la antigüedad, pero no crea stock ni capacidad y no debe presentarse como garantía absoluta de servicio.

Referencia de implementación: PlanificadorPedidos.ordenar, seleccionar y prioridadEfectiva; MotorSimulacion.cambiarEstrategia.

Tiempo simulado y objetivos de servicio

TiempoSimulado representa la duración real del trabajo por unidad. EscalaTiempoSimulado transforma el tiempo monotónico para decisiones de planificación y objetivos. Son responsabilidades distintas, aunque sus nombres puedan sugerir lo contrario.

```
duración real del trabajo = unidades totales × ms por unidad
tiempo simulado transcurrido = tiempo real transcurrido × factor
```

El reloj guarda un origen real y una fuente LongSupplier, System.nanoTime por defecto. Las pruebas pueden inyectar una fuente controlada. Se limita el resultado al máximo de long y se excluyen transcurridos negativos. La escala queda fija por instancia.

Servicio	Prioridad interna	Objetivo de espera
Exprés	5	Cualitativo casi inmediato
Prioritario	4	10 minutos simulados
Estándar	3	30 minutos simulados
Programado	2	60 minutos simulados
Económico	1	120 minutos simulados

El motor mide espera de servicio desde la aceptación hasta después de adquirir recursos y consumir stock, al comenzar preparación. Incluye espera por mercancía y equipo. serviceWaitAtStart se fija una sola vez. Si es menor o igual al objetivo, el resultado es DENTRO; si lo supera, EXCEDIDO. Antes del inicio se publica ESPERANDO; Exprés usa CUALITATIVO.

La escala afecta también envejecimiento. En comparación, el intervalo de 80 ms es simulado; llegadas de 5 ms, trabajo, límite y métricas continúan en tiempo real. Aumentar factor no acelera la preparación de pedidos.

Referencia de implementación: TiempoSimulado.duracionMillis; EscalaTiempoSimulado; NivelServicio; MotorSimulacion.workerLoop y snapshot.

Formación y detección de deadlock

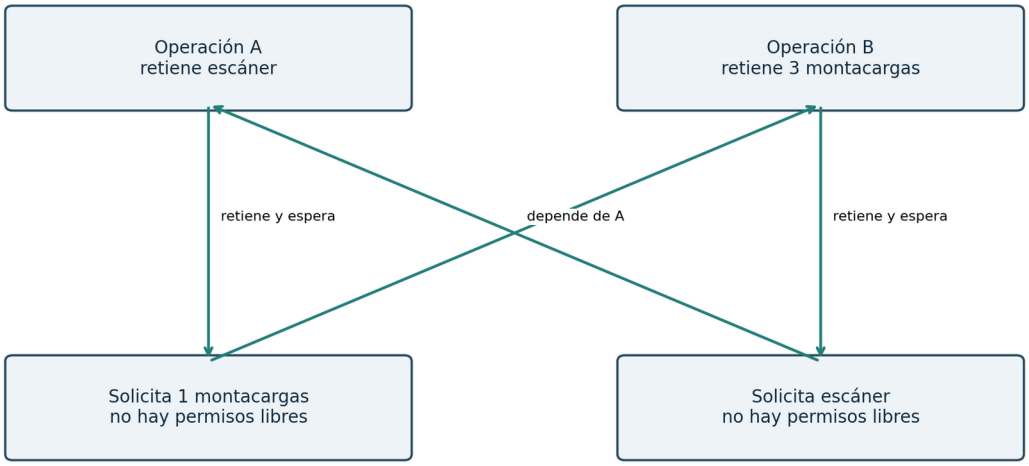


Diagrama 4. Espera circular del escenario didáctico con recursos reales.

DemostracionDeadlock coordina dos operaciones mediante CountDownLatch(2). Cada una adquiere su primer recurso, reduce el contador y espera que ambas hayan llegado. Solo después solicita el segundo recurso; así no depende de acertar pausas para formar el conflicto.

A conserva 1 escáner y pide 1 montacargas. B conserva los 3 montacargas y pide 1 escáner. Retener los tres es necesario porque el administrador no identifica unidades individuales; retener solo uno dejaría otros disponibles para A.

GestorRecursos registra cada espera y sus propietarios esperados. findCycle recorre ese grafo con DFS y esperarCiclo espera una notificación de cambio. El ciclo es concluyente en este escenario aislado, donde están agotados ambos pools. Un grafo de propietarios de recursos con múltiples instancias no debe generalizarse sin análisis adicional a un detector universal de deadlock.

Referencia de implementación: DemostracionDeadlock.operate y coordinate; GestorRecursos.esperarCiclo, findCycle y dfs.

Resolución y aislamiento del conflicto

El coordinador pausa Recepcion y el despacho del motor, comprueba recursos libres y entra en una sesión exclusiva del administrador. Las órdenes normales aceptadas se conservan. El controlador exige generador detenido y evita que este envíe nuevos pedidos durante el escenario.

1. resolver utiliza compareAndSet para aceptar una sola elección cuando el conflicto está confirmado.

2. El coordinador interrumpe el hilo desplazado. try-with-resources libera sus reservas y doneA o doneB confirma que ese intento salió.

3. La operación elegida obtiene lo faltante, ejecuta su trabajo y libera recursos.

4. recover vuelve a ejecutar la desplazada con adquisición conjunta en el orden seguro.

5. El coordinador espera limpieza y restaura recepción y despacho.

Condiciones de Coffman

Exclusión mutua: escáner y permisos son finitos. Retener y esperar: cada operación conserva un recurso mientras pide otro. No expropiación: el administrador no arrebata permisos; la operación interrumpida los devuelve al salir. Espera circular: A depende de B y B de A. El funcionamiento normal rompe la circularidad mediante un orden global.

Alcance exacto de la recuperación

recover solicita 1 escáner y 1 montacargas para cualquiera de las operaciones. Por tanto, si se desplaza B, el reintento no conserva su requerimiento inicial de 3 montacargas. La versión demuestra liberación y reintento seguro, pero esa diferencia debe reconocerse al explicar el escenario. No representa una reejecución idéntica de una carga que siempre necesite tres equipos.

La demostración no reserva ni modifica stock. cancelar interrumpe operaciones y coordinador; el cierre emplea esperas acotadas. La liberación se realiza por las propias reservas y no mediante releases arbitrarios desde JavaFX.

Referencia de implementación: DemostracionDeadlock.resolver, recover, cancelar, close y coordinate; GestorRecursos.entrarModoDemostracion.

Generador continuo de pedidos

GeneradorPedidos recibe el motor, el registro de clientes y observadores de errores y aceptación. Configuracion valida intervalo mínimo de 1 ms y rango de unidades positivo. iniciar impide tareas duplicadas y usa scheduleAtFixedRate en un ScheduledExecutorService de un hilo.

DataSequence crea o reutiliza una cuenta sintética por nivel. Los niveles siguen el ciclo del enum; no se sortean por semilla. Random elige el total de unidades, mezcla productos y reparte cantidades entre una y hasta tres líneas, sin superar el total. Cada Pedido genera un UUID nuevo y entra por MotorSimulacion.enviar con origen AUTOMATICO.

La semilla reproduce productos y cantidades para la misma configuración. Los niveles son cíclicos y las cuentas se derivan del nivel, no de la semilla. UUID, marcas de tiempo y orden concurrente no se reproducen. Esta precisión debe preferirse a descripciones resumidas que atribuyen todos los campos a Random.

Un error de envío se informa y la tarea continúa en la próxima llegada. No hay reposición automática de stock. LongAdder acumula aceptados y distribución; detener no cancela pedidos ya admitidos.

Detención

detener marca inactivo, llama a shutdown y espera hasta 30 segundos. Si hace falta, usa shutdownNow y espera otros 5 segundos. Restaura la interrupción del hilo llamador y elimina la referencia al scheduler. El controlador realiza esta espera fuera del hilo JavaFX.

La llegada continua no tiene límite de negocio, pero orders, reservas de historial y registros crecen en memoria. El índice del ciclo de niveles es int; una ejecución extremadamente prolongada también requiere atender su eventual desbordamiento. La configuración se captura al iniciar, no se aplica en caliente.

Referencia de implementación: GeneradorPedidos.iniciar, detener, close y DataSequence.siguiente; OrigenPedido.

Comparación controlada de políticas

EscenarioComparacion es un record inmutable. Los perfiles visibles contienen 15 o 20 pedidos. La semilla mezcla servicios balanceados y el catálogo; cada pedido programado contiene un producto y 1 a 3 unidades. IDs lógicos CMP-001 en adelante se convierten en UUID deterministas con nameUUIDFromBytes.

Parámetro	Valor común
Workers	2
Trabajo por unidad	20 ms reales
Intervalo entre llegadas	5 ms reales
Envejecimiento	80 ms simulados
Límite por política	15000 ms reales por defecto
Almacén	5 ubicaciones de 100 unidades de volumen
Cola de recepción del experimento	8 mensajes
Stock inicial	Suma exacta de productos del escenario

ComparadorPoliticas.ejecutar evita comparaciones simultáneas en la misma instancia. runOne crea motor, inventario, almacén, recursos y recepción nuevos para cada política y las ejecuta secuencialmente. Primero carga y verifica stock y guarda la ocupación base de recursos; luego comienza la ventana medida.

Se programan llegadas relativas. El envío efectivo puede retrasarse por el planificador del SO; la espera de cada pedido comienza cuando el motor lo acepta. La ejecución espera todos los envíos y estados terminales de los UUID concretos, además de recursos libres; una cola vacía no basta.

fingerprint describe las entradas compartidas y no es un hash criptográfico. Estado distingue COMPLETA, CANCELADA, LIMITE_TIEMPO, FALLIDA y NO_EJECUTADA. Las filas parciales conservan conteos sin publicar tiempos como si fueran una ejecución completa.

Referencia de implementación: EscenarioComparacion.generar y fingerprint; ComparadorPoliticas.ejecutar, runOne y verifyInitialInventory.

Cálculo e interpretación de métricas

Para cada pedido se registran llegada, inicio efectivo y finalización en nanosegundos reales. Las duraciones se convierten a milisegundos dividiendo entre un millón. No deben interpretarse estas marcas monotónicas como fecha u hora civil.

Medida	Cálculo
Espera	Inicio efectivo menos llegada.
Procesamiento	Finalización menos inicio efectivo.
Permanencia	Finalización menos llegada.
Duración total	Última finalización menos inicio de reproducción.
Rendimiento	Completados dividido entre duración total en segundos.
Utilización de recurso	Permiso por tiempo ocupado dividido entre capacidad por ventana observada.

La espera media usa pedidos con inicio registrado; procesamiento requiere inicio y fin; permanencia usa finalizados. Las medias sin muestras son null y se muestran como ausentes. Los errores internos se distinguen en los conteos mediante el campo error aunque el estado operativo del motor sea CANCELADO.

En el modo interactivo, la utilización se observa desde la construcción del motor. En comparación se resta la ocupación de preparación inicial y se utiliza una ventana propia hasta confirmar recursos libres. La ventana de utilización puede ser ligeramente mayor que la duración hasta la última finalización.

Las métricas por servicio ayudan a detectar que una media global oculta esperas de niveles bajos. No existe un ganador universal. El escáner retenido durante todo el trabajo, las llegadas muy rápidas y el pequeño tamaño de las muestras condicionan las conclusiones. Repetir semillas o escenarios conserva entradas equivalentes, no tiempos idénticos.

Referencia de implementación: GestorRecursos.updateOccupied; MotorSimulacion.snapshot; ComparadorPoliticas.calcularMetricas y calculate.

Interfaz JavaFX y cierre del proceso

RedXelaApp carga redxela.fxml, aplica redxela.css y abre una escena de 1366 por 768 con tamaño mínimo de 900 por 600. FXML define BorderPane, navegación, contenido y mensaje inferior. RedXelaController crea las páginas y controles, conecta servicios y mantiene el borrador de líneas.

Un Timeline consulta snapshots cada 350 ms. Las operaciones breves de presentación se ejecutan en JavaFX Application Thread. Los comandos potencialmente bloqueantes se envían al executor redxela-ui-tareas; Platform.runLater entrega cambios visuales. La comparación se encapsula en Task con progreso y mensaje observables.

El refresco es periódico, no una garantía de visualización de cada transición corta. Un pedido de pocas unidades puede terminar entre refrescos. Las copias inmutables impiden que una tabla modifique directamente orders, reservas o propietarios.

Secuencia de cierre implementada

1. El manejador intercepta el cierre una sola vez, solicita cancelar la comparación, detiene Timeline y deshabilita la ventana.

2. Un hilo redxela-cierre cierra y espera el executor de tareas, con escalamiento a shutdownNow si excede el tiempo.

3. Cierra el generador y después Recepcion, Inventario, DemostracionDeadlock y MotorSimulacion.

4. El motor cancela pendientes, devuelve reservas no consumidas e interrumpe workers. Inventario interrumpe su consumidor y espera su salida.

5. Se restaura la señal de interrupción cuando corresponde y se solicita cerrar Stage desde JavaFX.

Los tiempos de cierre son acotados; el estado cierreCompleto debe comprobarse en pruebas. No basta asumir que un executor terminó por recibir shutdownNow. El callback Application.stop únicamente detiene Timeline; la coordinación completa reside en el cierre interceptado de la ventana.

Referencia de implementación: RedXelaApp.start y stop; RedXelaController.initialize y closeAsynchronously; MotorSimulacion.close.

Pruebas y evidencia de validación

El código contiene 49 métodos anotados con @Test en 10 clases JUnit. La documentación del proyecto registra mvn clean verify con 49 aprobadas, sin fallos, errores ni omisiones. Ese resultado corresponde a la ejecución reportada durante el desarrollo; el ZIP no incluye reportes Surefire para verificarla de manera independiente.

Suite	Casos	Aspectos cubiertos
ModeloTest	3	Validación de datos, unidades y catálogo.
PlanificadorPedidosTest	3	Orden, desempates y envejecimiento.
GestorRecursosTest	2	Capacidad, cierre idempotente e interrupción parcial.
AlmacenTest	3	Ubicaciones no contiguas y factibilidad.
InventarioTest	10	Cola, duplicados, reservas, espacio y coherencia.
MotorSimulacionTest	7	Despacho, cancelación, stock, límites y carga.
GeneradorYClientesTest	5	Cuentas, semilla, IDs y detención.
DemostracionDeadlockTest	7	Ciclo, recuperación, aislamiento y cancelación.
ComparadorPoliticasTest	6	Entradas, métricas, aislamiento y limpieza.
ObjetivosServicioTest	3	Escala, objetivos y resultado al inicio.

La reproducción oficial consiste en mvn clean verify desde la raíz y revisar target/surefire-reports. Para cambios de concurrencia se deben mantener límites de espera, coordinación explícita y aserciones sobre recursos e hilos; evitar exigir que una política siempre gane por milisegundos.

Las capturas de la sesión muestran funciones de la interfaz y resultados concretos, pero no prueban todas las rutas de error ni el cierre del proceso. docs/PRUEBAS_MANUALES.md define el recorrido integrado; registrar su ejecución requiere observar cada resultado.

Referencia de implementación: src/test/java; README.md; docs/REVISION_REQUISITOS.md; docs/PRUEBAS_MANUALES.md.

Decisiones y límites de la versión

Decisiones adoptadas

El estado en memoria simplifica el enfoque en Sistemas Operativos, a costa de perder datos al cerrar. Los pools acotados evitan un hilo por pedido. La reserva completa evita preparación parcial. La asignación determinista facilita reproducir ubicaciones. El escenario aislado de deadlock protege el inventario del modo operativo.

Límites técnicos relevantes

1. El escáner único, retenido durante toda la preparación, limita la concurrencia útil. Separar fases sería una extensión, no un comportamiento presente.

2. El almacén no compacta y una recepción se aplica completa. Puede faltar espacio compatible aun con volumen libre; tampoco se garantiza avance de todo pedido que cabe en un almacén vacío.

3. recover usa un montacargas para reintentar B, frente a los tres iniciales. Debe conservarse esa distinción en la defensa o corregirse antes de afirmar recuperación idéntica.

4. Cancelar un pedido ya despachado interrumpe su hilo. workerLoop restaura la interrupción y retorna, sin enviar un nuevo bucle de trabajador. En una sesión larga, esas cancelaciones pueden reducir los bucles disponibles para atender pedidos posteriores.

5. El historial, cuentas sintéticas, mensajes y órdenes consumidas permanecen en memoria. No hay persistencia, compactación de historial ni un límite global de solicitudes pendientes por espacio.

Los callbacks y cierres incluyen manejo de errores, pero algunas rutas auxiliares silencian excepciones, como closeQuietly en la demostración. Las comprobaciones de terminación y disponibilidad son evidencia necesaria; no constituyen una prueba formal de ausencia de todos los bloqueos posibles.

La separación interfaz y dominio es útil, aunque RedXelaController concentra muchas responsabilidades. La auditoría incluida tiene algunas referencias antiguas a métodos como Almacen.asignar o consumir; en esta versión los métodos efectivos son planificar, aplicar y extraer.

Referencia de implementación: CatalogoMercancias; MotorSimulacion.cancelar y workerLoop; DemostracionDeadlock.recover y closeQuietly; Almacen.